SafeDrop: A Single-Executable Home-Hosted Secure File Drop with Reachability Evidence and Proof-Carrying Delivery

Draft research article for a first IJRC / IJRCom-style submission

Author placeholder: Your Name Here | Affiliation placeholder: Your Institution / Independent Research | Corresponding author: your-email@example.com

Article Type	Keywords	SDG Alignment	Status
Research Article / Systems Design	secure file sharing; home hosting; verifiable delivery	SDG 9 and SDG 16	Submission-ready first draft

Abstract

SafeDrop is a design for a single-executable secure file drop server that can be run on a home PC, laptop, or small edge device without a cloud account. The system addresses two problems that are usually treated separately: making a personal host reachable from the public Internet when it is online, and producing portable evidence that a shared file was delivered intact and on time. SafeDrop combines content-addressed storage, append-only audit logs, expiring capability links, signed transfer receipts, automated reachability selection, privacy-aware metadata controls, and an explanation-first admin console. The paper contributes a layered architecture, a reachability descriptor model, an evidence pipeline that can later be wrapped by succinct proofs, and a 50-track research roadmap that separates productizable work from frontier experiments. Rather than report invented deployment numbers, this paper defines an explicit implementation sequence and evaluation protocol for future validation on commodity laptops, Raspberry Pi-class devices, phones, and household routers. The central claim is that strong delivery evidence and practical home-hosted usability are compatible if the system is built around signed state transitions, conservative path selection, and human-readable trust surfaces.

Keywords: secure file sharing; home hosting; verifiable delivery; NAT traversal; audit log; post-quantum migration; local-first systems

1. Introduction

Consumer and small-team file sharing is dominated by cloud platforms that are operationally convenient but structurally centralized. In the ordinary case these platforms solve link generation, public reachability, and transfer reliability, but they also centralize metadata, place operational trust in a third party, and often provide little durable evidence beyond transient logs or dashboard messages. At the same time, self-hosted alternatives remain difficult for ordinary users because the hardest operational problem is not running a local web server; it is making a home machine reliably reachable through dynamic IP addresses, NATs, firewalls, sleeping devices, and restrictive ISP policies.

The resulting gap is larger than a usability issue. It is also a systems and security gap. Users who need a file to remain under local control often need more than confidentiality; they need proof-oriented answers to questions such as whether a link was reachable, whether the recipient obtained the same file that was offered, whether a shared capability expired on time, and whether a disputed transfer can be reconstructed without trusting a single mutable application log. Existing consumer systems rarely treat these questions as first-class protocol outcomes.

SafeDrop is proposed as a response to that gap. The project is intentionally narrow: it is not a collaboration platform, not a messaging suite, and not a distributed cloud storage replacement. It is a single-executable file drop and share server that an individual can run on a laptop, workstation, mini PC, or Raspberry Pi. The design objective is to let a user publish a secure link from a home device whenever that device is online, transfer files with bounded operational friction, and export evidence that the transfer and link lifecycle were handled correctly.

The architectural novelty of SafeDrop lies in combining home-hosted reachability, proof-carrying delivery, privacy-aware metadata controls, and explanation-first UX inside one local-first system. The design assumes that succinct zero-knowledge proofs are scientifically attractive but operationally expensive. For that reason the

baseline system is built on signed state transitions, append-only audit structures, and deterministic manifests. Succinct proofs are treated as wrappers over already-useful evidence rather than as a prerequisite for basic correctness. This sequencing is important for solo development and for honest scientific reporting.

The paper makes five contributions. First, it defines a layered system architecture for a one-binary file sharing server that is suitable for commodity personal hardware. Second, it introduces a reachability descriptor model that records which network path was selected for a share session and why. Third, it proposes a proof-carrying evidence pipeline in which signed receipts, Merkle checkpoints, and export bundles provide practical assurance even before advanced proving systems are added. Fourth, it connects privacy controls, energy-aware operation, and explainable admin UX to the file-sharing problem instead of treating them as separate product embellishments. Fifth, it organizes the broader SafeDrop agenda into fifty research tracks so the platform can evolve into multiple publishable studies without collapsing the MVP into frontier complexity.

Contributions

- A layered architecture for single-executable, home-hosted secure file sharing.
- A reachability descriptor that records how a recipient actually reached the host.
- A proof-carrying evidence model based on audit events and signed receipts.
- A systems methodology that separates MVP implementation from frontier research tracks.

2. Related Work and Motivation Gap

SafeDrop sits at the intersection of secure transport, NAT traversal, tamper-evident logging, proof systems, privacy-preserving systems, and local-first software. QUIC provides a modern transport substrate with strong security and multiplexing semantics that are attractive for both direct transfer and relay-based forwarding [1]. For connectivity establishment across consumer networks, the ICE, STUN, and TURN ecosystem remains the dominant practical toolkit for traversing NATs and selecting candidate paths [2]-[4]. These standards, however, do not themselves provide a portable evidence model that explains later which path was used for a given file-sharing transaction.

For integrity and tamper evidence, Merkle trees remain a natural foundation because they provide compact inclusion proofs and straightforward append semantics [5]. Public transparency systems demonstrate how append-only, checkpointed logs can support external verification and accountability [6]. SafeDrop adapts that mindset to personal file sharing: instead of publishing certificates, it publishes evidence about manifest commitments, link issuance, path selection, receipt acknowledgments, and expiry state transitions.

The privacy-related literature offers multiple ingredients but not a direct personal-server recipe. Differential privacy formalizes how metadata or analytics surfaces can be blurred under an explicit privacy budget [7]. Oblivious RAM demonstrates how access-pattern leakage can be reduced under stronger adversarial assumptions, albeit with substantial overhead [8]. Conflict-free replicated data types provide a path for multi-device consistency and eventually convergent sync [9]. WebAssembly contributes a practical sandboxing model for user-defined policy modules [13]. SafeDrop borrows from all of these areas while keeping the baseline deployment model intentionally small.

The proving and post-quantum landscape is also relevant. Modern succinct proof systems such as Halo and PLONK make it realistic to imagine optional proof wrappers over delivery or relay traces [10], [11]. At the same time, the current post-quantum transition has shifted from speculative planning to concrete standards: NIST has finalized ML-KEM for key establishment and ML-DSA for digital signatures, alongside SLH-DSA as a hash-based signature option [14]-[16]. That development matters directly to SafeDrop because exported delivery evidence may need to remain verifiable over long time horizons.

What the literature still leaves open is a productively narrow question: how should one design a home-hosted secure file drop that can be used by non-expert operators, is reachable from commodity networks when online, produces exportable evidence, and remains architecturally ready for stronger proofs and post-quantum migration? SafeDrop is positioned as a systems answer to that question.

3. Research Questions and Design Goals

The project is guided by three research questions. **RQ1** asks whether a home device can expose a secure public sharing link with minimal operator intervention whenever the device is online. **RQ2** asks whether delivery, expiry, and deletion state transitions can be exported as tamper-evident evidence that is useful before any advanced proving machinery is attached. **RQ3** asks whether the same system can remain lightweight enough

to run as a single executable on commodity personal hardware while still offering privacy controls and energy-aware behavior.

These questions lead to several design goals. The first goal is **zero-account deployability**: the system should not require a cloud identity or external control plane to function. The second goal is **path transparency**: if the system chooses public bind, router mapping, assisted direct connection, or relay fallback, that path choice must become part of the evidence record. The third goal is **proof-friendly determinism**: manifests, events, and capability semantics must be deterministic so that later proof systems can wrap existing artifacts instead of forcing redesign. The fourth goal is **human-usable trust**: the admin console must explain what happened in ordinary language, not only as hashes. The fifth goal is **graceful degradation**: advanced research tracks must remain optional and feature-gated.

Just as important are the non-goals. SafeDrop is not trying to hide all traffic metadata from a globally powerful adversary in its first implementation; it aims to reduce exposure while preserving usability. It is not claiming perfect physical deletion from consumer hardware; instead, it focuses on crypto-shredding plus auditable deletion state transitions. It is also not trying to replace mature cloud collaboration suites. The narrower scope is deliberate because scientific credibility depends on resisting the temptation to claim a universal secure-sharing platform before the core lifecycle is sound.

Design Goal	Why It Matters	MVP Enforcement
Zero-account deployability	Keeps SafeDrop meaningfully local-first and usable by non-admin users	No mandatory cloud control plane or recipient account
Path transparency	Explains whether delivery was direct, mapped, or relayed	Signed reachability descriptor exported with the share
Proof-friendly determinism	Allows later succinct proofs to wrap existing artifacts	Deterministic manifests and versioned events
Human-usable trust	Makes evidence useful during disputes and audits	Admin console evidence cards plus export bundle
Graceful degradation	Prevents frontier features from breaking the product	Feature flags and strict sequencing

Table 1. SafeDrop design goals and the mechanisms that keep them enforceable during implementation.

4. Methods

4.1 System architecture and assumptions

SafeDrop models the share lifecycle as a sequence of immutable or append-only state transitions. A file is first chunked and stored in a content-addressed store. A manifest records ordered chunk references, total size, hash algorithm choice, content type, and any alias metadata needed for the outward sharing surface. The manifest is committed to an audit structure, producing a stable root that later events can reference. A share object then binds the manifest to an expiry, recipient scope, path preferences, and download constraints.

The architecture is layered to make implementation and verification tractable. The protocol facade handles web pages, REST APIs, capability URLs, resumable transfer, and public status surfaces. Below it sits a content-addressed storage core that makes manifests deterministic and proof-friendly. A dynamic access engine is responsible for deciding how a recipient can actually reach the host. A verifiability layer records events, issues signed receipts, and prepares export bundles. A privacy and policy layer enforces metadata minimization, optional Wasm policies, and recipient-scoped access choices. Finally, an admin and explanation console translates low-level evidence into understandable status cards.

This layered arrangement is not merely an engineering convenience. It is how the project keeps its research ambition from destabilizing the product. Every frontier idea must fit behind an interface that preserves the baseline semantics of manifests, receipts, or path descriptors. For example, a later zero-knowledge circuit for delivery should consume signed receipt chains rather than redefine what counts as delivery. Likewise, a future relay honesty proof should be about a transcript that the base relay already records for auditing purposes.

The single-executable requirement affects multiple implementation choices. Rust is a practical fit because it supports memory-safe systems code, predictable async services, and static linking on mainstream targets. A reasonable workspace split is still useful internally, but the release artifact should bundle the needed modules into one deployable binary. Embedded assets for the admin console can be compiled into the release. Optional relay mode may still require a second deployment role, but the core owner-host experience remains intentionally simple: one executable, one data directory, one admin entry point.

Figure 1 summarizes the resulting layer contract. The important point is that each layer exposes artifacts that higher layers can explain and lower layers can verify. This is how SafeDrop tries to stay publishable as a system design while also remaining buildable by a solo engineer using AI agents.

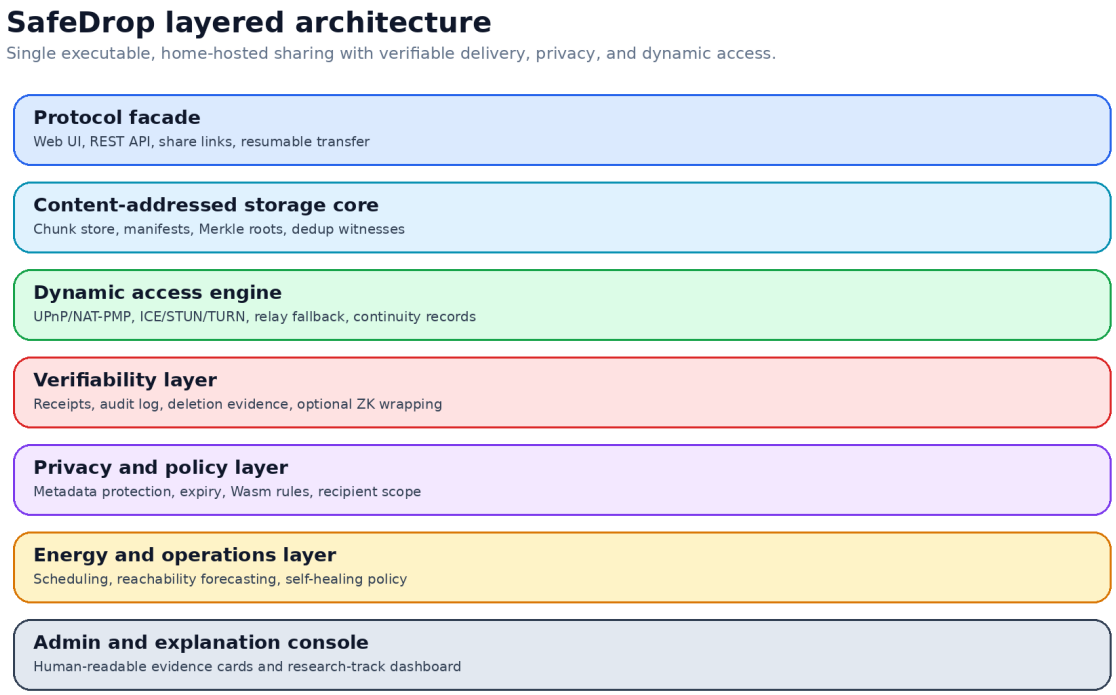

Figure 1. SafeDrop layered architecture used throughout the paper and the implementation plan.

4.2 Reachability protocol

The central operational challenge is public reachability from consumer networks. SafeDrop therefore treats path selection as a first-class protocol decision. When a share is created, the dynamic access engine attempts paths in a strict order. If the host is already publicly reachable, the system uses that direct address. If not, it tries automatic router mapping through UPnP or NAT-PMP. If mapping either fails or cannot be externally confirmed, the system may attempt assisted direct connectivity through ICE-style candidate negotiation. Only then does it allocate an encrypted relay path.

The output of this process is a **reachability descriptor**. This descriptor binds the share or share family to a path type, an epoch, the host identity key, and continuity metadata. If the host address changes, the descriptor can be re-issued as a signed update without pretending that the IP address is the stable identity of the system. The stable object is the host key and its continuity chain. This design matters because a recipient or later auditor needs to understand whether a file was delivered directly, through a mapped router port, through assisted direct connectivity, or through an encrypted relay.

In the baseline design, claims of successful reachability are grounded in signed probes rather than abstract cryptographic magic. After requesting a router mapping, SafeDrop performs an external verification step. The result is not a universal proof that any network on Earth can reach the host, but it is a concrete, logged statement that a designated probe succeeded for the advertised endpoint during a given epoch. This is a more honest and implementable claim, and it creates a clean boundary for later formalization or proof compression.

The relay path is intentionally narrower than a VPN or general-purpose proxy. The relay only forwards encrypted transfer traffic and minimal session control metadata. It should not terminate file-layer encryption or access plaintext payloads. The research roadmap later explores transcript commitments and relay honesty proofs, but the first operational win is simpler: preserve end-to-end confidentiality, record when relay fallback was used, and ensure that relay dependence is visible rather than hidden.

Figure 2 shows the path-selection logic. The system always prefers the path with the least additional infrastructure, but it never treats path choice as incidental. In SafeDrop, the selected path becomes part of what can later be explained and, eventually, proved.

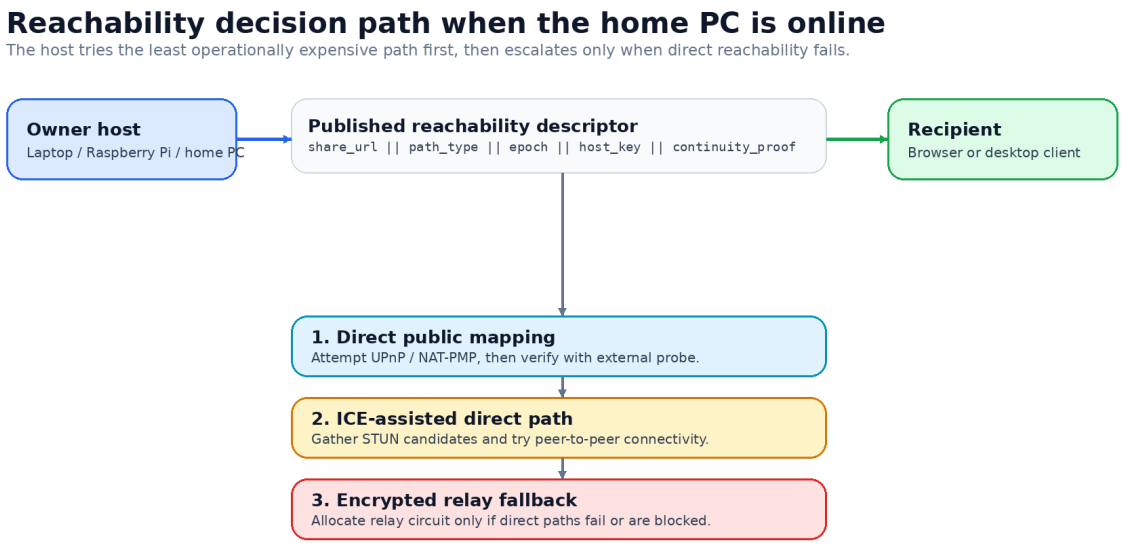

Figure 2. Path-selection flow for the home-hosted share lifecycle.

4.3 Evidence model and proof-carrying delivery

The evidence model begins with an append-only audit log that records high-value state transitions. At minimum these include manifest commitment, share issuance, reachability confirmation, download start, segment acknowledgments, final receipt confirmation, link expiry, and deletion events. Each event is inserted into a Merkle-backed structure with periodic signed checkpoints. The audit log serves three purposes: it allows internal consistency checking, produces exportable witness paths, and establishes the stable semantics that later succinct proof systems may consume.

Delivery evidence is intentionally layered. The first layer is the manifest root, which identifies what was offered. The second layer is a sequence of signed transfer receipts that can be emitted per segment or per resumable checkpoint. The third layer is a final completion receipt acknowledging that the manifest corresponding to a specific root has been fully transferred. This construction is already useful without any zero-knowledge component because it supports dispute reconstruction, resumable transfer semantics, and independent verification by the sender or another reviewer.

Optional zero-knowledge wrapping is then framed as a research enhancement rather than a baseline dependency. A future prover could compress the receipt chain into a succinct statement that a valid sequence of segment acknowledgments culminated in full delivery of the committed manifest. The key architectural point is that the validity conditions already exist before the proof system is added. This sequencing lowers product risk and makes scientific claims easier to evaluate, because the project can separately measure the usefulness of signed receipts and the incremental cost of succinct proofs.

Deletion and expiry are treated with the same restraint. An expiry event is fully deterministic: once the capability reaches its expiry epoch, new access should be rejected and the revocation should appear in the log. Deletion is harder because consumer hardware rarely supports perfect physical erasure claims. SafeDrop therefore proposes envelope encryption with per-object key destruction, followed by auditable deletion state transitions and garbage-collection evidence where available. The system can honestly prove that the object became cryptographically unrecoverable under the managed keys, even if it cannot prove a laboratory-grade media wipe.

Evidence is not complete unless it is interpretable. The admin console should therefore derive concise evidence cards from the same events: what file root was shared, which path was used, when delivery completed, whether expiry occurred, whether relay fallback was needed, and what artifacts are exportable. This explanation layer is not marketing decoration. It is what turns low-level proofs into operational trust.

4.4 Privacy, policy, and energy-aware controls

Privacy in SafeDrop is not reduced to transport encryption. Several distinct surfaces require separate treatment: public link metadata, recipient identity signals, storage layout leakage, telemetry, and operator-facing analytics. The first implementation should distinguish between canonical internal metadata and outward-facing presentation metadata. That separation allows aliases, size bucketing, and coarse timestamps to be used on public pages without corrupting the immutable internal manifest. When stronger guarantees are needed, the roadmap includes privacy-budget accounting for metadata surfaces and optional access-pattern reduction modes for small sensitive workloads.

Policy is treated as a deterministic layer rather than a collection of ad hoc UI toggles. The baseline system can express ordinary constraints such as link expiry, download counts, recipient scope, relay permission, and storage retention. A more advanced track uses WebAssembly modules for custom policies, but only after the host interface is sufficiently small and deterministic. Determinism matters because any policy decision that cannot later be reconstructed or explained undermines the evidence model.

Energy-aware behavior is included because the owner host is often a laptop or intermittently powered small device rather than a rack server. Large transfers, proof generation, or relay-heavy sessions can have noticeable energy cost. SafeDrop therefore proposes a scheduling layer that can defer non-urgent work, batch heavier cryptographic operations, and surface the current operating mode to the user. The principle is not to trade away minimum security for convenience, but to expose approved performance profiles and to prefer postponement over silent weakening.

Explainability bridges the privacy and operations story. For instance, a user should be able to see whether a shorter expiry was suggested because the file resembled a previous high-sensitivity share, or whether relay fallback was preferred because past direct attempts repeatedly failed. Any on-device AI or heuristics remain advisory by default. The operator remains in control, and the console should show why a suggestion was made.

4.5 Implementation sequence

The implementation plan is designed for solo execution with AI-agent assistance. Phase 0 creates the repository, vocabulary, website, and document spine. Phase 1 delivers the thin vertical slice: chunk ingestion, manifest commitment, share issuance, resumable transfer, and audit logging. Phase 2 focuses on reachability: router mapping, signed external probe, path descriptor generation, and encrypted relay fallback. Phase 3 adds export bundles, explanation cards, hybrid post-quantum interfaces, and reproducible build artifacts. Only after these stages does the project begin frontier work such as relay honesty proofs, ORAM-style access hiding, or succinct proof wrappers.

This sequencing is not a compromise on novelty; it is a methodology for achieving novelty without losing operational truthfulness. A large class of security projects fail because they present the hardest cryptographic feature as the core system identity, then never complete the product scaffolding required to evaluate it in realistic settings. SafeDrop instead tries to make every early milestone both demonstrable and publishable. For example, a first paper can legitimately focus on home-hosted reachability with signed continuity evidence even before any zero-knowledge layer exists.

The accompanying research roadmap expands this stance into fifty tracks. Those tracks are organized into bundles for home reachability, relay trust, proof-carrying sharing, private edge sharing, post-quantum and supply-chain trust, and runtime and UX foundations. This bundling strategy allows a project to generate multiple future papers without overloading the first prototype or the first submission.

Bundle	Initial Track Set	Publication Angle
Home reachability	R06, R08, R09, R35, R39	Practical reachability from consumer networks
Proof-carrying sharing	R10, R11, R27, R28	Delivery, expiry, and export evidence
Relay trust	R07, R22, R33	Relay fallback and forwarding accountability
Private edge sharing	R03, R15, R31, R43	Metadata minimization and evidence exports
PQ and supply chain	R12, R21, R41, R42, R50	Long-lived trust and release integrity
Runtime and UX	R05, R44, R47	Operability on constrained personal

		hardware

Table 2. Bundle structure for the expanded 50-track SafeDrop research agenda.

5. Results: Analytical Findings and Validation Criteria

Because this paper is a systems-design article rather than a completed deployment report, the results presented here are analytical and methodological rather than empirical throughput numbers. Even so, the design yields several concrete findings. First, reachability can be represented as a bounded set of explicit path states with stable semantics. This is already an advance over ordinary consumer file-sharing tools, which often hide whether transfer occurred directly, through a cloud proxy, or through a fallback service. In SafeDrop, the selected path is recorded and exportable.

Second, useful delivery evidence does not need to wait for succinct zero-knowledge proofs. The combination of deterministic manifests, segment receipts, final completion receipts, and Merkle-backed event logging already creates a practical audit chain. This matters because it turns the question of advanced proof systems into a cost-benefit optimization problem instead of an all-or-nothing prerequisite. The system can therefore be evaluated in stages: first on evidence usefulness, then on proof compression overhead.

Third, privacy and energy controls are architecturally separable from core transfer correctness. Metadata aliasing, budgeted privacy surfaces, scheduling policies, and advisory models can be added without changing what a manifest is or what counts as delivery. That separation reduces implementation risk and improves scientific clarity because the baseline lifecycle can be studied independently from optional policy layers.

Fourth, the single-executable requirement is compatible with a research-grade architecture as long as module boundaries are kept internal and release artifacts remain externally simple. The project does not require microservices to be academically interesting. In fact, keeping the deployment model small is part of the research claim: verifiable, privacy-aware sharing should be accessible to users who would never deploy a cluster.

These analytical results lead directly to falsifiable validation criteria. The first criterion is operational reachability: most sessions on permissive home networks should complete through direct bind or automated router mapping without manual configuration, while more restrictive networks should fall back cleanly to relay. The second criterion is evidence coherence: an exported bundle should let an external reviewer reconstruct the manifest, the selected path, the completion event, and the expiry state without hidden mutable dependencies. The third criterion is usability: the owner should be able to interpret evidence cards without reading cryptographic source code. The fourth criterion is efficiency: the baseline evidence model should add only modest overhead to real file transfer on commodity hardware.

The project should therefore be evaluated on laptops, Raspberry Pi-class devices, Android recipients, and at least one household router configuration. Micro-benchmarks should record upload speed, resumable transfer latency, receipt-generation cost, energy consumption per gigabyte, and relay overhead. Macro scenarios should include ordinary freelancer document exchange, small-team handoff of revision packages, and journalist-style source material transfer. Success should be judged not merely by raw speed, but by whether the resulting evidence is complete, portable, and comprehensible.

Evaluation Axis	Measurement Focus	Success Condition
Reachability	Direct bind or automated mapping success; fallback cleanliness	Most permissive home networks work without manual router editing; restrictive networks relay cleanly
Evidence coherence	Manifest, path, completion, expiry, and deletion state can be reconstructed	External reviewer can validate exported bundle without hidden mutable state
Usability	Owner interpretation of evidence cards and exports	Non-expert can explain what happened and what remains uncertain
Efficiency	Transfer overhead, receipt cost, energy per GB	Evidence overhead remains modest on laptops and small boards
Research extensibility	Ease of adding PQ, ZK, and privacy	Frontier tracks consume stable artifacts

	layers	rather than forcing redesign

Table 3. Validation matrix used to judge whether the first SafeDrop prototype is scientifically and operationally credible.

6. Discussion

SafeDrop's biggest strength is also its main methodological burden: it attempts to unite systems usability and formal evidence instead of treating them as separate publication lanes. That makes the project scientifically interesting, but it also requires discipline. The paper therefore argues strongly against over-claiming. In particular, one should not promise proof of universal reachability, perfect deletion on consumer hardware, or fully anonymous traffic against a global adversary in the first public system. Those are different research problems with different assumptions.

The relay honesty track illustrates this tension. A privacy-preserving relay that forwards only ciphertext is practical and valuable early. A relay that produces a succinct proof of correct forwarding is far more ambitious. Treating both as the same milestone would be a mistake. The first should become part of the product baseline if direct reachability is not possible. The second should remain a frontier publication track that is layered on top of an already useful relay design.

The same caution applies to privacy. Differentially private metadata presentation, recipient-scoped links, and optional access-pattern reduction are all worthwhile, but they incur usability or performance costs. SafeDrop should expose these trade-offs explicitly in the admin console and in exported evidence. Hiding the cost model would make the system harder to trust, not easier.

Finally, the paper argues that explainability is a scientific requirement, not only a UX refinement. If a system claims to produce proof of delivery, non-expert operators must be able to answer what was proved, under which path assumptions, and with what residual limitations. Without that layer, advanced cryptography risks becoming an opaque badge rather than a trustworthy instrument.

7. Conclusion

SafeDrop proposes a realistic but ambitious path toward home-hosted secure file sharing with strong evidence semantics. Its central thesis is that a one-binary local-first server can become both practically reachable and cryptographically accountable if the system treats manifests, path selection, receipts, expiry, and explanation as co-equal design objects. The baseline evidence model relies on deterministic manifests, signed receipts, and Merkle checkpoints; it does not depend on frontier proof systems to be useful.

The broader fifty-track roadmap shows how the project can grow after the MVP without losing coherence. In that sense SafeDrop is both a product plan and a publication strategy. The first implementation target is clear: build the usable share lifecycle and home reachability path first, then layer stronger proofs, privacy modes, and post-quantum migration on top of a system that already works.

Declarations

AI Usage Statement. AI tools were used to assist with drafting structure, editing, and website scaffolding. The author remains responsible for the technical claims, system design, and final manuscript content.

Data Availability Statement. This design paper does not rely on a released empirical dataset at the current stage. Any future benchmark datasets, traces, or anonymized artifacts will be released with the prototype evaluation where feasible.

Code Availability Statement. A public repository named SafeDrop is intended for the prototype implementation. The code accompanying the first prototype should include build instructions, schema versions, and evidence-export examples.

Funding Statement. No external funding is declared for this stage of the project.

Conflict of Interest Declaration. The author declares no competing financial or personal interests related to this work.

Ethical Considerations. No human or animal subjects were involved in this design study. Future user studies or deployment studies will require informed consent and appropriate ethical review where applicable.

References

[1] J. Iyengar and M. Thomson, RFC 9000: QUIC: A UDP-Based Multiplexed and Secure Transport, IETF, 2021.

[2] J. Rosenberg et al., RFC 8445: Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal, IETF, 2018.

[3] M. Petit-Huguenin et al., RFC 8489: Session Traversal Utilities for NAT (STUN), IETF, 2020.

[4] R. Mahy et al., RFC 8656: Traversal Using Relays around NAT (TURN), IETF, 2020.

[5] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in Advances in Cryptology - CRYPTO ’87, 1988.

[6] B. Laurie, A. Langley, and E. Kasper, RFC 9162: Certificate Transparency Version 2.0, IETF, 2021.

[7] C. Dwork and A. Roth, The Algorithmic Foundations of Differential Privacy. Boston, MA: Now Publishers, 2014.

[8] E. Stefanov et al., "Path ORAM: An Extremely Simple Oblivious RAM Protocol," in CCS, 2013.

[9] M. Shapiro et al., "Conflict-Free Replicated Data Types," in SSS, 2011.

[10] S. Bowe, J. Grigg, and D. Hopwood, "Halo: Recursive Proof Composition without a Trusted Setup," IACR ePrint 2019/1021, 2019.

[11] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive Arguments of Knowledge," IACR ePrint 2019/953, 2019.

[12] J. O’Connor et al., "BLAKE3: One Function, Fast Everywhere," 2020.

[13] A. Haas et al., "Bringing the Web up to Speed with WebAssembly," in PLDI, 2017.

[14] National Institute of Standards and Technology, FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard, 2024.

[15] National Institute of Standards and Technology, FIPS 204: Module-Lattice-Based Digital Signature Standard, 2024.

[16] National Institute of Standards and Technology, FIPS 205: Stateless Hash-Based Digital Signature Standard, 2024.

[17] L. Lamport, Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley, 2002.

[18] M. Bellare and S. K. Miner, "A Forward-Secure Digital Signature Scheme," in CRYPTO, 1999.

[19] S. Torres-Arias et al., "in-toto: Providing farm-to-table guarantees for bits and bytes," in USENIX Security, 2019.